# Assignment 7: Code Refactoring

```
import java.util.*;

class BankingSystem {

    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.createAccount("SAVINGS", "Rahul", 1000);
        bank.createAccount("CURRENT", "Amit", 5000);
        bank.createAccount("BUSINESS", "Neha", 10000);

        bank.deposit("Rahul", 500);
        bank.withdraw("Rahul", 200);
        bank.transfer("Rahul", "Amit", 100);

        bank.applyInterest("Rahul");
        bank.applyInterest("Neha");

        bank.sendNotification("Rahul");
        bank.generateReport();
    }
}

class Bank {

    private Map<String, Account> accounts = new HashMap<>();
```

```
public void createAccount(String type, String name, double balance) {
    if (type.equals("SAVINGS")) {
        accounts.put(name, new SavingsAccount(name, balance));
    } else if (type.equals("CURRENT")) {
        accounts.put(name, new CurrentAccount(name, balance));
    } else if (type.equals("BUSINESS")) {
        accounts.put(name, new BusinessAccount(name, balance));
    } else {
        System.out.println("Invalid account type");
    }
}

public void deposit(String name, double amount) {
    Account acc = accounts.get(name);
    acc.balance += amount;
    logTransaction("DEPOSIT", name, amount);
}

public void withdraw(String name, double amount) {
    Account acc = accounts.get(name);

    if (acc instanceof SavingsAccount) {
        if (acc.balance - amount < 500) {
            System.out.println("Minimum balance violation");
            return;
```

```
        }
    } else if (acc instanceof CurrentAccount) {
        if (acc.balance - amount < -1000) {
            System.out.println("Overdraft exceeded");
            return;
        }
    } else if (acc instanceof BusinessAccount) {
        if (acc.balance - amount < -5000) {
            System.out.println("Business overdraft exceeded");
            return;
        }
    }

    acc.balance -= amount;
    logTransaction("WITHDRAW", name, amount);
}

public void transfer(String from, String to, double amount) {
    Account a1 = accounts.get(from);
    Account a2 = accounts.get(to);

    if (a1.balance < amount) {
        System.out.println("Insufficient funds");
        return;
    }
```

```
        a1.balance -= amount;
        a2.balance += amount;

        logTransaction("TRANSFER_OUT", from, amount);
        logTransaction("TRANSFER_IN", to, amount);
    }

    public void applyInterest(String name) {
        Account acc = accounts.get(name);

        if (acc instanceof SavingsAccount) {
            acc.balance += acc.balance * 0.05;
        } else if (acc instanceof CurrentAccount) {
        } else if (acc instanceof BusinessAccount) {
            acc.balance += acc.balance * 0.02;
        }
    }

    public void sendNotification(String name) {
        System.out.println("Email sent to " + name);
        System.out.println("SMS sent to " + name);
        System.out.println("Push notification sent to " + name);
    }

    private void logTransaction(String type, String name, double amount) {
        System.out.println("LOG " + type + " " + amount + " " + name);
```

```
        System.out.println("Saved to file");
        System.out.println("Saved to database");
    }

    public void generateReport() {
        System.out.println("Report Start");
        for (Account acc : accounts.values()) {
            System.out.println(acc.name + " -> " + acc.balance);
        }
        System.out.println("Export PDF");
        System.out.println("Export Excel");
        System.out.println("Export CSV");
    }
}

class Account {
    String name;
    double balance;

    Account(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }
}

class SavingsAccount extends Account {
```

```
    SavingsAccount(String name, double balance) {
        super(name, balance);
    }
}

class CurrentAccount extends Account {
    CurrentAccount(String name, double balance) {
        super(name, balance);
    }
}

class BusinessAccount extends Account {
    BusinessAccount(String name, double balance) {
        super(name, balance);
    }
}

interface BankingOperations {
    void deposit();
    void withdraw();
    void applyLoan();
    void calculateTax();
    void openAccount();
    void closeAccount();
}
```

```
class ATM implements BankingOperations {

    public void deposit() {
        System.out.println("ATM deposit");
    }

    public void withdraw() {
        System.out.println("ATM withdraw");
    }

    public void applyLoan() {
    }

    public void calculateTax() {
    }

    public void openAccount() {
    }

    public void closeAccount() {
    }
}

class MobileApp implements BankingOperations {

    public void deposit() {
```

```
        System.out.println("Mobile deposit");
    }

    public void withdraw() {
        System.out.println("Mobile withdraw");
    }

    public void applyLoan() {
        System.out.println("Loan via mobile");
    }

    public void calculateTax() {
        System.out.println("Tax calculation");
    }

    public void openAccount() {
        System.out.println("Open account mobile");
    }

    public void closeAccount() {
        System.out.println("Close account mobile");
    }
}

class LoanService {
```

```
    private Bank bank = new Bank();

    public void applyLoan(String name, double amount) {
        bank.deposit(name, amount);
        System.out.println("Loan applied " + name);
    }

    public void approveLoan(String name) {
        System.out.println("Loan approved " + name);
    }

    public void rejectLoan(String name) {
        System.out.println("Loan rejected " + name);
    }
}

class TaxService {

    public void calculate(Account acc) {
        if (acc instanceof SavingsAccount) {
            System.out.println("Tax savings");
        } else if (acc instanceof CurrentAccount) {
            System.out.println("Tax current");
        } else if (acc instanceof BusinessAccount) {
            System.out.println("Tax business");
        }
```

```
    }
}

class NotificationService {

    public void email(String name) {
        System.out.println("Email " + name);
    }

    public void sms(String name) {
        System.out.println("SMS " + name);
    }

    public void push(String name) {
        System.out.println("Push " + name);
    }
}

class ReportService {

    public void pdf(List<Account> list) {
        System.out.println("PDF done");
    }

    public void excel(List<Account> list) {
        System.out.println("Excel done");
```

```
    }

    public void csv(List<Account> list) {
        System.out.println("CSV done");
    }
}
```

---

**Section 1: Code Smell Identification**

**Question 1**

**List all responsibilities handled by the Bank class.**
**Which violate the Single Responsibility Principle (SRP)?**

---

**Question 2**

**Identify all usages of if-else and instanceof.**
**Why do these violate the Open/Closed Principle (OCP)?**

---

**Question 3**

**Identify tightly coupled parts of the system.**
**Explain how they violate the Dependency Inversion Principle (DIP).**

---

**Question 4**

**Analyze the BankingOperations interface.**
**Which methods are unnecessary for ATM and MobileApp?**
**Which principle is violated?**

---

**Question 5**

**Check whether subclasses of Account follow Liskov Substitution Principle (LSP).**
**Explain your reasoning.**

---

**Section 2: Manual Refactoring (Without LLM)**

**Question 6**

**Manually refactor the Bank class to follow SRP.**
**List the new classes and their responsibilities.**

---

**Question 7**

**Manually remove instanceof usage from withdraw() using polymorphism.**
**Redesign the Account hierarchy.**

---

**Question 8**

**Refactor account creation logic manually using a suitable design pattern.**
**Explain your design.**

---

**Question 9**

**Split the BankingOperations interface manually to follow ISP.**
**Define the new interfaces.**

---

**Question 10**

**Refactor LoanService manually to follow Dependency Inversion Principle.**
**Show how dependencies are injected.**

---

**Section 3: LLM-Based Refactoring**

**Question 11**

**Write a prompt to refactor the Bank class using SRP.**
**Use an LLM and generate the refactored code.**

---

**Question 12**

**Write a prompt to remove instanceof using polymorphism.**
**Generate and analyze the output.**

---

**Question 13**

**Use an LLM to redesign account creation using Factory Pattern.**
**Provide the prompt and generated solution.**

---

**Question 14**

**Use an LLM to refactor the BankingOperations interface into smaller interfaces.**
**Evaluate the response.**

---

**Question 15**

**Use an LLM to apply Dependency Injection in LoanService.**
**Show the generated code.**

---

**Section 4: Comparison (Manual vs LLM)**

**Question 16**

**Compare manual refactoring vs LLM-generated refactoring for SRP.**
**Which approach produced cleaner design and why?**

---

**Question 17**

**Compare how polymorphism was implemented manually vs by LLM.**
**Which solution is more extensible?**

---

**Question 18**

**Evaluate the correctness of LLM-generated Factory Pattern implementation.**
**Did it fully follow OCP?**

---

**Question 19**

**Compare interface segregation done manually vs via LLM.**
**Which version avoids unnecessary methods better?**

---

**Question 20**

**Compare dependency injection implementations.**
**Which version is more flexible and testable?**

---

**Section 5: Advanced LLM Thinking**

**Question 21**

**Did the LLM-generated code introduce any new design issues?**
**Identify and explain.**

---

**Question 22**

**How many iterations (prompts) were needed to get a good design from the LLM?**
**What improvements were made in each iteration?**

---

**Question 23**

**What instructions/prompts improved the LLM output the most?**

---

**Question 24**

**Can LLM fully replace manual design thinking in this case?**
**Justify your answer.**

---

**Section 6: Reflection**

**Question 25**

**Which approach (manual vs LLM) gave you deeper understanding of SOLID principles?**

---

**Question 26**

**In real-world projects, when should you rely on LLM and when should you not?**

---

**Question 27**

**What are the risks of blindly using LLM-generated refactoring?**